CMake for Beginners

A Complete Step-by-Step Guide

Master CMakeLists.txt from scratch

1. What is CMake?

CMake is a **cross-platform build-system generator**. You describe your project in a file called `CMakeLists.txt`, and CMake generates the actual build files (Makefiles on Linux, Visual Studio projects on Windows, Xcode projects on macOS) for you.

Why CMake?	
Write once, build anywhere	One CMakeLists.txt works on Linux, Windows, and macOS
Handles dependencies	Automatically finds libraries installed on your system
Industry standard	Used by OpenCV, LLVM, Qt, and thousands of open-source projects
IDE integration	VS Code, CLion, and Visual Studio all understand CMake natively

2. Installing CMake

Platform	Command / Method
Ubuntu / Debian	`sudo apt install cmake`
Fedora / RHEL	`sudo dnf install cmake`
macOS (Homebrew)	`brew install cmake`
Windows	`Download installer from cmake.org/download`
Verify install	`cmake --version`

3. Typical Project Structure

A well-organised CMake project keeps source files separate from build output. Here is the recommended layout:

```
my_project/

CMakeLists.txt # <-- the main CMake file (root level)

src/

main.c

utils.c

include/

utils.h

build/ # <-- generated by CMake (never edit)
```

Golden rule: always build in a separate folder (out-of-source build). This keeps generated files away from your source code and lets you delete the build folder cleanly without touching your code.

4. Anatomy of a CMakeLists.txt

Every CMakeLists.txt is built from a handful of commands. Below is a complete minimal example followed by a line-by-line explanation.

Complete minimal example

```
cmake_minimum_required(VERSION 3.15)


project(HelloWorld # project name

VERSION 1.0 # optional: set version number

LANGUAGES C) # languages used: C, CXX, ASM ...


# Create an executable called 'hello' from main.c

add_executable(hello src/main.c)


# Tell the compiler where to find header files

target_include_directories(hello PRIVATE include/)
```

5. Commands Explained One by One

cmake_minimum_required()

```
cmake_minimum_required(VERSION 3.15)
```

This **must be the very first line** of every CMakeLists.txt. It tells CMake the oldest version that can build your project. Use 3.15 or higher for modern features.

project()

```
project(MyApp VERSION 2.0 LANGUAGES C CXX)
```

Key parameters:

Parameter	Example	What it does
Name	`MyApp`	Sets PROJECT_NAME variable
VERSION	`VERSION 2.0`	Sets PROJECT_VERSION (major.minor)
LANGUAGES	`LANGUAGES C CXX`	Enables C and C++ compilers
DESCRIPTION	`DESCRIPTION "..."`	Sets PROJECT_DESCRIPTION string

add_executable()

```
add_executable(my_app # name of the binary to produce

src/main.c # list all .c / .cpp files

src/utils.c

src/math_helpers.c)
```

This is the most common command. It tells CMake: *"compile these source files and link them into an executable called* `my_app`*."* List every `.c` or `.cpp` file that belongs to the binary.

add_library()

```
# Static library (.a / .lib)

add_library(mathlib STATIC src/math.c src/trig.c)


# Shared / dynamic library (.so / .dll)

add_library(mathlib SHARED src/math.c src/trig.c)


# Header-only (no compiled source)

add_library(headers INTERFACE)
```

Use `add_library` when you want to build reusable code that other targets (executables or libraries) can link against. **STATIC** embeds the code at link time. **SHARED** produces a separate file loaded at runtime.

target_link_libraries()

```
target_link_libraries(my_app # the target that needs the library

PRIVATE # visibility keyword (see below)

mathlib # a library built by this project

m # system library: libm (math)
```

```
pthread) # system library: pthreads
```

Keyword	Meaning
`PRIVATE`	Only this target uses the library. Not exposed to targets that link against this target.
`PUBLIC`	This target and anyone linking against it both get the library.
`INTERFACE`	Only targets that link against this target get the library (not this target itself).

target_include_directories()

```
target_include_directories(my_app

PRIVATE include/ # your own headers

PUBLIC external/somelib/inc # headers callers also need

)
```

Tells the compiler where to search for `#include` files. Use the same PRIVATE/PUBLIC/INTERFACE keywords as `target_link_libraries`. Prefer **PRIVATE** unless the headers need to be visible to other targets.

6. Variables

```
# Set a variable

set(MY_VAR "hello")

set(SRC_FILES src/main.c src/utils.c)


# Read a variable with ${}

message(STATUS "Value: ${MY_VAR}")

add_executable(my_app ${SRC_FILES})


# Useful built-in variables

message(STATUS "Source dir: ${CMAKE_SOURCE_DIR}")

message(STATUS "Build dir: ${CMAKE_BINARY_DIR}")

message(STATUS "Project: ${PROJECT_NAME}")
```

Variable	What it contains
`CMAKE_SOURCE_DIR`	Absolute path to the top-level CMakeLists.txt folder
`CMAKE_BINARY_DIR`	Absolute path to the build folder
`CMAKE_CURRENT_SOURCE_DIR`	Source dir of the currently-processed CMakeLists.txt
`PROJECT_NAME`	Name given to project()
`PROJECT_VERSION`	Version string given to project()
`CMAKE_C_COMPILER`	Path to the C compiler being used
`CMAKE_BUILD_TYPE`	Debug / Release / RelWithDebInfo / MinSizeRel

7. Compiler Flags and Build Types

```
# Set the C standard

set(CMAKE_C_STANDARD 11) # use C11
```

```
set(CMAKE_C_STANDARD_REQUIRED ON) # error if compiler can't do C11


# Add compiler warnings per-target (preferred over global flags)

target_compile_options(my_app PRIVATE

-Wall -Wextra -Wpedantic # GCC/Clang warning flags

)


# Build type — pass on command line:

# cmake -DCMAKE_BUILD_TYPE=Release ..

if(CMAKE_BUILD_TYPE STREQUAL "Debug")

target_compile_definitions(my_app PRIVATE DEBUG_MODE=1)

endif()
```

Build Type	Flags (GCC/Clang)	Use when
Debug	`-g -O0`	Developing / debugging
Release	`-O3 -DNDEBUG`	Shipping to users
RelWithDebInfo	`-O2 -g -DNDEBUG`	Profiling
MinSizeRel	`-Os -DNDEBUG`	Embedded / size-constrained

8. Conditionals, Loops, and Options

```
# Boolean option — user can toggle with -DENABLE_TESTS=ON

option(ENABLE_TESTS "Build unit tests" OFF)


if(ENABLE_TESTS)

message(STATUS "Tests enabled")

add_subdirectory(tests/)
```

```
endif()


# Platform check

if(WIN32)

target_link_libraries(my_app PRIVATE ws2_32)

elseif(UNIX AND NOT APPLE)

target_link_libraries(my_app PRIVATE pthread)

endif()


# Loop over a list

set(SOURCES a.c b.c c.c)

foreach(SRC ${SOURCES})

message(STATUS "Source file: ${SRC}")

endforeach()
```

9. Finding External Libraries

CMake ships with hundreds of **Find modules** that locate common libraries automatically.

```
# Find OpenSSL (comes bundled with CMake)

find_package(OpenSSL REQUIRED) # REQUIRED = error if not found

target_link_libraries(my_app PRIVATE OpenSSL::SSL OpenSSL::Crypto)


# Find a library manually if no Find module exists

find_library(MYLIB_PATH # result variable

NAMES mylib libmylib # possible names to look for
```

```
PATHS /usr/local/lib ~/libs # extra search paths

)

if(NOT MYLIB_PATH)

message(FATAL_ERROR "mylib not found!")

endif()

target_link_libraries(my_app PRIVATE ${MYLIB_PATH})
```

10. Splitting a Project with add_subdirectory()

Large projects split their code into sub-folders, each with its own `CMakeLists.txt`.

Folder layout

```
my_project/
CMakeLists.txt # root: calls add_subdirectory()
src/
CMakeLists.txt # builds the main executable
lib/
CMakeLists.txt # builds a helper library
tests/
CMakeLists.txt # builds and registers tests
```

Root CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(MyProject LANGUAGES C)

add_subdirectory(lib) # processes lib/CMakeLists.txt first
add_subdirectory(src) # can use targets defined in lib/

option(ENABLE_TESTS "Run tests" OFF)
if(ENABLE_TESTS)
add_subdirectory(tests)
endif()
```

lib/CMakeLists.txt

```
add_library(mylib STATIC helper.c logging.c)

target_include_directories(mylib PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)
```

src/CMakeLists.txt

```
add_executable(my_app main.c)

target_link_libraries(my_app PRIVATE mylib) # link against lib/
```

11. Building Your Project — Step by Step

Step 1 — Create a build folder

```
mkdir build && cd build
```

Keep build files separate from source code.

Step 2 — Configure

```
cmake ..
```

CMake reads CMakeLists.txt and generates build files. The .. means 'the source is one level up'.

Step 3 — Build

```
cmake --build .
```

Compiles and links everything. Add --parallel 4 to use 4 CPU cores.

Step 4 — Run

```
./my_app
```

On Windows: my_app.exe

Optional — Set build type

```
cmake -DCMAKE_BUILD_TYPE=Release ..
```

Pass options with -D before the source path.

Optional — Clean

```
cmake --build . --target clean
```

Or simply delete the entire build/ folder.

12. Complete Real-World Example

This example shows a realistic CMakeLists.txt for a C project with a library, an executable, compiler warnings, build-type flags, an optional test build, and a platform check all in one file.

```
cmake_minimum_required(VERSION 3.15)
project(SensorApp
VERSION 1.2.0
LANGUAGES C)

# ■■ C standard ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

# ■■ Helper library ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
add_library(sensor_lib STATIC
lib/sensor.c
lib/filter.c
)
target_include_directories(sensor_lib PUBLIC lib/include)
target_compile_options(sensor_lib PRIVATE -Wall -Wextra)

# ■■ Main executable ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
add_executable(sensor_app src/main.c src/cli.c)
target_include_directories(sensor_app PRIVATE src/include)
```

```
target_link_libraries(sensor_app PRIVATE sensor_lib)

target_compile_options(sensor_app PRIVATE -Wall -Wextra -Wpedantic)


# ■■ Platform-specific libs ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

if(WIN32)

target_link_libraries(sensor_app PRIVATE ws2_32 psapi)

elseif(UNIX)

target_link_libraries(sensor_app PRIVATE m pthread)

endif()


# ■■ Optional tests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

option(BUILD_TESTS "Build unit tests" OFF)

if(BUILD_TESTS)

enable_testing()

add_subdirectory(tests)

endif()


# ■■ Install rules ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

install(TARGETS sensor_app DESTINATION bin)

install(TARGETS sensor_lib DESTINATION lib)
```

13. Common Errors and How to Fix Them

Error message	Cause	Fix
CMake Error: Could not find cmake_minimum_required	cmake_minimum_required() is not the first line	Move it to line 1, above even comments
undefined reference to ...	Missing library in target_link_libraries()	Add the missing library name

No such file or directory (header)	Include path not set	Add target_include_directories() with the right folder
CMake Error: The source ... does not appear to contain CMakeLists.txt	Wrong source path passed to cmake	Make sure you point cmake to the folder with CMakeLists.txt
target_link_libraries called on non-existent target	Typo in target name or wrong order of add_subdirectory()	Check spelling; ensure the library subdirectory is added before the

14. Quick Reference Cheat Sheet

Command	Purpose
`cmake_minimum_required(VERSION x.y)`	Set minimum CMake version (line 1)
`project(Name LANGUAGES C CXX)`	Name your project and pick languages
`add_executable(target a.c b.c)`	Build an executable
`add_library(lib STATIC a.c)`	Build a static library
`add_library(lib SHARED a.c)`	Build a shared library
`target_link_libraries(t PRIVATE lib)`	Link a library to a target
`target_include_directories(t P dir)`	Add header search paths
`target_compile_options(t P -Wall)`	Add compiler flags
`target_compile_definitions(t P FOO)`	Add preprocessor defines
`set(VAR value)`	Set a CMake variable
`option(OPT "desc" OFF)`	User-togglable boolean option
`if() / elseif() / else() / endif()`	Conditional logic
`add_subdirectory(dir)`	Include another CMakeLists.txt
`find_package(Pkg REQUIRED)`	Locate an installed library
`enable_testing() / add_test()`	Register tests for ctest
`install(TARGETS t DESTINATION bin)`	Define install rules
`message(STATUS "...")`	Print a message during configure
`cmake --build . --parallel 4`	Build using 4 parallel jobs
`cmake -DCMAKE_BUILD_TYPE=Release ..`	Configure for Release build

Next steps: explore cmake.org/documentation for the full command reference. The cmake-examples repository on GitHub has dozens of real-world project templates to learn from.